État d'avancement ? ERP

Système de gestion pour compagnie de transport de voyageurs ? 15 août 2026
Stack : Django + DRF (backend) · Angular standalone (frontend) · multi-tenant (sociétés / agences) · i18n FR/EN

1. Vision & périmètre

ERP multi-sociétés (multi-tenant) pour une compagnie de transport : gestion de la flotte, des voyages, des réservations/billetterie, des colis et bagages, de la tarification, ainsi que l'administration (sociétés, agences, utilisateurs, rôles et permissions). Chaque société (institution) est isolée ; un super-administrateur plateforme provisionne les sociétés ; chaque société a un administrateur d'institution qui gère son personnel.
Modules métier à venir (specs fournies ultérieurement) : Trésorerie, Ressources Humaines (RH), Comptabilité.

2. Architecture des modules

Backend ? apps Django (apps/)

? core ? socle (permissions, réponses, pagination, audit, devises).
? authentication ? utilisateur custom, JWT, 2FA, sessions, me/, me/permissions.
? system_config ? sociétés, agences, packages, abonnements, paramètres, feature-flags.
? admin_rbac ? modules, menus, permissions, rôles, profils, affectations (c?ur RBAC).
? users ? gestion des utilisateurs (institution).
? fleet ? flotte (véhicules, marques, modèles, conducteurs, carburant, maintenance, documents, missions, accidents, parkings, fournisseurs, contrats, config).
? voyages_trips ? voyages (cycle de vie : soumission, départ, arrivée, embarquement, annulation, transbordement, équipages).
? voyages_booking ? réservations, passagers, billets, retours, embarquement, feuille de route.
? voyages_config ? config voyages (villes, points, trajets, plans de bus, catégories tarifaires, créneaux, colis, fidélité, modèle de billet, paramètres).
? voyages_consignment ? colis & bagages (réception, manifeste, livraison, impressions).
? voyages_pricing ? tarifs.
? hr ? RH (modèles présents, vues minimales ? à compléter).
? treasury ? Trésorerie (modèles présents, vues minimales ? à compléter).
? sync_scig_sage ? synchronisation Sage / SCIG (inventaires, zones, transferts).
? online_booking ? réservation en ligne (public).

Frontend ? Angular (angular/src)

Pages métier sous _api/pages/* (fleet, fleet-config, voyages, voyages-booking, voyages-config, voyages-consignment, voyages-pricing, admin, rbac, system, users, scig). Services sous _api/services/*. Menu latéral piloté par app/core/service/sidebar/sidebar.service.ts.

3. Travaux réalisés (session en cours)

Corrections « mise en place d'une entreprise »

? Seed des devises (table Currency vide ? création d'entreprise échouait) : migration core/0003.
? Scoping société à la création d'agences, rôles et profils par un admin d'institution (isolation ; plus de champ 'company' requis).
? Identité de l'entreprise (RCCM, forme juridique, NIU, CNPS, TVA, capital) : exposée par le serializer liste et éditable ? ne s'écrase plus à l'enregistrement.
? me/ et login renvoient 'company' comme objet (CompanyRef) et 'is_superuser' ? corrige companyId nul (assignation de profils) et le gating du menu.

? Écrans multi-institutions réservés au super-admin (marqueurs de menu 'super_admin').

Autres correctifs

? Création de bagage : tolérance des champs optionnels vides/null (serializer robuste + reset de formulaire complet).
? Page d'attribution des permissions repensée : hiérarchie Module ? Ressource ? cases d'action (CRUD) avec sélection groupée.

4. RBAC ? état détaillé (base pour la suite)

Le contrôle d'accès basé sur les rôles est complet et effectif de bout en bout. Section clé pour intégrer les futurs modules (Trésorerie, RH, Comptabilité).

4.1 Mécanisme

? Backend : classe core.permissions.HasModulePermission. Chaque vue déclare permission_classes = [IsAuthenticated, HasModulePermission] + un permission_map = { 'GET': '<code>.view', 'POST'/'PATCH'/'PUT'/'DELETE': '<code>.manage' }.
? Ordre d'autorisation : super-admin (is_superuser) ? tout ; admin d'institution (is_institution_admin) ? tout dans sa société (bypass) ; sinon vérifie le code via get_user_permissions(user, société).
? Frontend : directive standalone *hasPermission='code' (ou tableau, au moins un) dans app/shared/directives/ ; masque menus et boutons. Réactive au chargement des permissions.
? Menu : filtrage générique dans sidebar.service.ts/buildMenuForUser ? toute entrée avec un champ 'permission' est masquée si non accordée ; groupes/modules vides masqués ; super/admin institution bypass via currentUser.
? me/permissions renvoie ['*'] pour super-admin ET admin d'institution ; sinon les codes des rôles/profils. PermissionService (front) charge ce endpoint au login/bootstrap.

4.2 Taxonomie des codes de permission

Format : <module>.<ressource>.<action>. 102 permissions seedées, reproductibles via migrations (admin_rbac 0002 à 0007). Préfixes de module de permission (? noms d'apps) :

? fleet.* (33) ? app fleet (+ fleet.config.* groupé pour la config de référence).
? voyage.* (11) ? app voyages_trips (trip, crew, dashboard).
? ticketing.* (17) ? apps voyages_booking (booking/customer/ticket) ET voyages_consignment (ticketing.parcel.*, ticketing.luggage.*).
? transport.* (14) ? app voyages_config (city/route/fareclass/terminal + transport.config.* groupé) ET voyages_pricing (transport.fare.*).
? scig.* (27) ? app sync_scig_sage.

4.3 Modules déjà protégés (backend + menu + boutons)

? Flotte (fleet) ? codes fleet.* ? 0 IsInstitutionAdmin restant.
? Voyages (voyages_trips) ? codes voyage.*.
? Réservations (voyages_booking) ? codes ticketing.*.
? Config Voyages (voyages_config) ? codes transport.* + transport.config.*.
? Colis & Bagages (voyages_consignment) ? codes ticketing.parcel.* / ticketing.luggage.*.
? Tarifs (voyages_pricing) ? codes transport.fare.*.

Vérifié : aucune vue de ces 6 modules n'utilise plus IsInstitutionAdmin ; tout est gouverné par les permissions.

4.4 Rôles & profils par défaut

À la création d'une société (seed_default_roles, appelé par _bootstrap_company), 6 rôles + 6 profils homonymes scopés à la société sont créés (idempotent) : Chef d'agence, Guichetier, Régulateur, Gestionnaire de flotte, Agent colis & bagages, Comptable (lecture). Commande de backfill : python manage.py seed_default_roles [--company <id>] (déjà exécutée sur la base de dev).

5. Recette : intégrer un NOUVEAU module au RBAC

Procédure reproductible (suivie pour les 6 modules). À appliquer pour Trésorerie, RH, Comptabilité une fois leurs specs et endpoints prêts :

? Définir les codes de permission du module : <module>.<ressource>.<action> (view/manage, ou create/update/delete si granularité utile ; un code groupé <module>.config.* pour la config de référence).

? Seeder ces codes via une data-migration admin_rbac (rattachée à un AppModule), sur le modèle de 0003-0007 ? indispensable pour que base fraîche/tests/prod les aient.

? Backend : sur chaque vue, remplacer IsInstitutionAdmin par [IsAuthenticated, HasModulePermission] + permission_map (GET?.view, écriture?.manage). Objectif : 0 IsInstitutionAdmin.

? Frontend menu : ajouter 'permission' sur chaque entrée de menu du module dans sidebar.service.ts.

? Frontend boutons : importer HasPermissionDirective et poser *hasPermission sur Nouveau/Modifier/Supprimer + actions ; envelopper dans <ng-container *hasPermission> les boutons déjà sous *ngIf.

? Tests pytest (patron _make_staff) : staff avec .view ? lecture OK / écriture 403 ; avec .manage ? écriture OK ; admin ? tout.

? Rôles par défaut : ajouter les nouveaux codes aux bundles pertinents dans admin_rbac/services/defaults.py (ex. Comptable pour la Compta/Trésorerie, un rôle RH dédié) + rejouer seed_default_roles.

6. Prochaines étapes ? Trésorerie, RH, Comptabilité

? Trésorerie (app treasury existante : modèles présents, vues minimales) ? compléter les endpoints puis appliquer la recette RBAC (codes ex. treasury.*).

? RH (app hr existante : modèles présents, vues minimales) ? idem (codes ex. hr.*), prévoir un rôle 'Responsable RH'.

? Comptabilité (à créer) ? nouvelle app + codes ex. accounting.*, rôle 'Comptable' enrichi.

? Pour chacun : brainstorm ? spec ? plan ? exécution (workflow superpowers), puis intégration RBAC via la recette (section 5), puis mise à jour des rôles par défaut.

? Interconnexions à prévoir : Réservations/Colis (encaissements) ? Trésorerie ; Trésorerie/Paie ? Comptabilité ; RH (employés) ? conducteurs/équipages (fleet/voyages).

7. Points d'attention / dette technique

? Après tout changement du payload utilisateur (is_superuser, company objet, permissions), les sessions ouvertes doivent se reconnecter (cache localStorage) ? sinon menu/permissions obsolètes.

? Les codes de permission historiques n'étaient dans aucune migration (présents seulement en dev) : corrigé (migrations 0003-0007). Toujours seeder par migration.

? Avertissement staticfiles (dossier static manquant) ? bénin, à nettoyer.

? Taxonomie de permission (voyage/ticketing/transport) ? noms d'apps (voyages_*) : documenté ici (section 4.2) pour éviter la confusion lors des extensions.

? Durcissement possible : guards de route Angular pour bloquer l'accès direct par URL aux écrans non autorisés (aujourd'hui : menu masqué + backend qui refuse).